

第一章

计算机设计目标:

性能 性价比 性能功耗比

性能测量

- 典型性能指标
 - 响应时间——事件开始到完成经过的时间
 - 吞吐量——在给定时间内完成得工作总量
- 机器X相对机器Y的加速比
$$Y \text{ 的执行时间} / X \text{ 的执行时间}$$
- 执行时间
 - 响应时间、逝去时间
 - 包含硬盘访问、内存访问、输入输出、操作系统时间开销等
 - CPU时间(针对服务器): 只包含计算时间, 不含IO操作时间开销
- Benchmark基准测试集
 - 核心指令集, 比如矩阵乘法
 - 玩具问题类程序, 比如Sorting
 - 合成的基准集, 比如Dhrystone
 - Benchmark套件, 比如SPEC06fp, TPC-C

汇总性能比较结果

SPECRatio比值：将执行时间对参考机器进行归一化，比值与机器性能成比例关系

$$\underline{SPECRatio} = \frac{Execution\ time_{reference}}{Execution\ time}$$

利用两个机器的SPECRatio比值来比较两个机器的性能。

$$\frac{SPECRatio_A}{SPECRatio_B} = \frac{\frac{Execution\ time_{reference}}{Execution\ time_A}}{\frac{Execution\ time_{reference}}{Execution\ time_B}} = \frac{Execution\ time_B}{Execution\ time_A} = \frac{Performance_A}{Performance_B}$$

注意：机器执行时间与机器性能成倒数关系。

计算分类：

SISD（单指令单数据流）

SIMD（单指令多数据流）：向量机 多媒体拓展处理 GPU；

MIMD（多指令多数据流）紧耦合 松散耦合；

弗林分类法

- SISD (single instruction stream, single data stream)单指令流单数据流
- SIMD (single instruction stream, multiple data stream)单指令流多数据流
 - 向量机
 - 多媒体扩展处理
 - GPU
- MIMD (multiple instruction stream, multiple data stream)多指令流多数据流
 - 紧耦合MIMD
 - 松散耦合MIMD
- MISD (multiple instruction stream, single data stream)多指令流单数据流
 - 无商业产品

晶体管动态功耗，含义

动态能耗与功率

- 单个晶体管的动态能耗
 - 晶体管状态转换 (1→0 或者 0→1) 所需要的能耗

$$\text{能耗}_{\text{动态}} \propto \text{容性负载} \times \text{电压}^2$$

- 晶体管的容性负载跟晶体管输出端金属连线电容、连接的其他晶体管数量和等效电容、制造工艺有关

- 单个晶体管的动态功率

$$\text{功率}_{\text{动态}} \propto \frac{1}{2} \times \text{容性负载} \times \text{电压}^2 \times \text{开关频率}$$

- 对一项工作，减少频率可以减少功率，不能减少能耗

例 现代一些微处理器设计采用可调电压，电压下降15%可能导致频率下降15%，这对动态能耗和动态功率有什么影响？

由于电容不变，能耗变化就是电压平方之比：

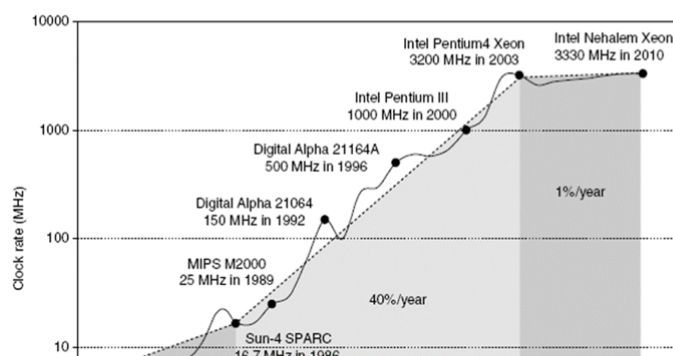
$$\frac{\text{能耗}_{\text{新}}}{\text{能耗}_{\text{原}}} = \frac{(\text{电压} \times 0.85)^2}{\text{电压}^2} = 0.85^2 = 0.72$$

能耗下降72%。对于功率，需要考虑频率变化

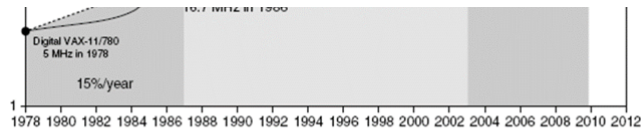
$$\frac{\text{新功率}}{\text{原功率}} = 0.72 \times \frac{\text{开关频率} \times 0.85}{\text{开关频率}} = 0.61$$

缩小为原来的61%

- Intel 80386 消耗~2W
- 3.3 GHz Intel Core i7 消耗 130W
- 热量必须从 $1.5 \times 1.5 \text{ cm}$ 芯片散发出去



● 130瓦是空气冷却的极限



进行状态转换的晶体管数量和转换频率的增加，超过了电容负载和电压的下降，导致了能耗和功耗的增加

能耗与容性负载*电压平方 正相关

单个晶体管还与开关率有关；

提升能耗效率

提升能耗：

- 提升能耗效率的方法
- 针对典型情况进行设计
 - PMD个人手持设备手机、平板电脑经常处于空闲状态，内存和存储设备提供低功耗模式来节约能耗
 - PC上微处理器利用集成在片上的温度传感器来自动检测温度调整工作活跃度，以避免CPU过热
- 超频
 - Turbo模式：在这种模式下，芯片可以判定在少数CPU核上以更高的频率运行小段时间是安全，直到温度开始上升。
 - 执行单线程代码，微处理器关掉多余的核，只留下一个CPU核并以更高主频(超过额定频率)运行

提升的方法：

- 1, 以逸待劳：关闭空闲模块
- 2, 调整低电压和时钟频率
- 3, 典型情况特殊设备处理
- 4, 超频 Turbo模式 与执行单线程代码

(含计算) 系统可靠性: MTBF MTTF MTTR计算

可信任度

- 系统模块的可信度由可靠性和可用性来衡量
- 模块可靠性: 从某个参考时刻开始的连续实现服务时间
 - MTTF (Mean time to failure): 平均无故障时间
MTTF的倒数是故障率, 经常用每十亿小时故障次数来衡量
 - MTTR (Mean time to repair): 故障平均修复时间, 服务中断度量
 - MTBF(Mean time between failures) 平均故障间隔时间= MTTF + MTTR
- 模块可用性: 当模块在服务完成与服务中断两种状态中切换, 服务完成占据的比率

$$\text{模块可用性} = \frac{MTTF}{MTTF + MTTR}$$

MTTF平均无故障时间 (meantime to failure)

MTTR故障平均修复时间 (Meantime to repair)

MTBF平均故障间隔时间 (Meantime between failure) =MTTF+MTTR

TF的倒数是故障率 系统故障率是各个组件故障率之和

例 设磁盘子系统组件和MTTF如下，求系统的MTTF。

10个磁盘，每个MTTF=1 000 000小时

1个ATA控制器，MTTF=500 000小时

1个电源，MTTF=200 000小时

1个风扇，MTTF=200 000小时

1根ATA电缆，MTTF= 1000 000小时

假设各个故障源互相独立，整个系统故障率是各个组件故障率之和

$$\begin{aligned}\text{系统故障率} &= 10 \times \frac{1}{1\,000\,000} + \frac{1}{500\,000} + \frac{1}{200\,000} + \frac{1}{200\,000} + \frac{1}{1\,000\,000} \\ &= \frac{10 + 2 + 5 + 5 + 1}{1\,000\,000} = \frac{23}{1\,000\,000} = \frac{23}{1\,000\,000\,000} \text{小时}\end{aligned}$$

系统的MTTF为故障率倒数：

$$\text{系统MTTF} = \frac{1}{\text{系统故障率}} = \frac{1\,000\,000\,000}{23\,000} = 43500 \text{小时}$$

例 磁盘子系统常常有备分冗余电源，以提高可信任度。在前面例子中系统增加一个备用电源，系统的MTTF变为多少？

假设系统组件发生故障相互独立。新系统(双电源系统，或者电源对系统)其中一个电源发生故障时，由于第二个电源的保障，系统依然能够正常工作。当一个电源发生故障，没有来得及修理，第二电源也发生故障，这时系统发生故障。

一个电源发生故障的概率P1(两个电源发生故障独立，故障率为单电源系统之和)：

$$\frac{2}{MTTF}$$

第一个电源发生故障，来不及修理第二个电源也坏了的概率P2：

$$\frac{MTTR}{MTTF}$$

修复时间MTTR越长，P2(第二次故障发生概率)越大；无故障时间MTTF越长，第二次故障概率越小。电源对系统的故障概率：

$$p1 \times p2 = \frac{2}{MTTF} \times \frac{MTTR}{MTTF}$$

所以电源对系统的MTTF为(故障率倒数)：

$$\frac{MTTF^2}{2 \times MTTR}$$

计算机设计遵循基本原则：

局部性原理，利用并行性，聚焦一般状况

并行性：多，吞吐，数据并行，性能

- 1, 多处理器多硬盘，流水线，多个部件；
- 2, 对于服务器，**将请求分配到不同的处理器和硬盘** 提升服务器吞吐率
- 3将数据分布存储到不同硬盘 以便并行读写数据 实现**数据的层次并行性**
- 4, 单个指令的**挖掘指令并行性**对高性能极为重要

计算机设计原理

- 利用并行性的优势
 - 比如多处理器、多个硬盘、流水线、多个功能部件
- 对于服务器，将请求平衡分发到不同的处理器和硬盘，提升服务器吞吐率
- 将数据分布存储到不同硬盘，以便并行读写数据，实现数据层次并行性 (data-level parallelism)
- 对于单个处理器，挖掘指令并行性 (instruction-level parallelism)，对获得高性能极为重要，比如利用流水线技术

局部性原理：最近指令，小部分代码，时间局部性，空间局部性，预测准确率

- 1程序趋于充分使用最近的指令和数据
- 2程序会花大量时间在小部分代码上
- 3有很高的预测准确率
- 4时间局部性：近期访问的不久会再次访问
- 5空间局部性：邻近的地址

计算机设计原理

● 局部性原理

- 程序趋向于充分使用最近用过的指令和数据
- 一个广泛认同的公理：程序会将90%的执行时间花费在仅仅10%的代码上。
- 根据程序近期访问来预测不久未来程序要访问的指令和数据，具有很高的预测正确率
- 时间局部性：近期被访问的数据和指令有可能在不久会被再次访问
- 空间局部性：地址邻近的数据和指令倾向于差不多同时被访问

聚焦一般情况：

考虑经常情况而不是特殊情况，

Amdahl定律，某些部件改进后整个系统加速比提升

计算机设计原理

● 聚焦于一般情况

- 在进行设计平衡时，优先考虑经常情况而不是特殊情况
- 在确定资源分配时，采用上述原则，因为优先考虑一般情况会带来更高的性能提升。
- **Amdahl定律**：描述系统的某些部件改进后(执行时间变少)，整个系统的加速比(性能提升)
- 该定律可用于衡量某些部件改进能提升多少系统性能，指导如何有效地分配资源以更好提升系统性能
- Amdahl定律可用于比较两种不同处理器设计方案的好坏

(含计算) 等效CPI，Amdahl定律计算：

Amdahl 定律

$$\text{加速比} = \frac{\text{整个任务未升级时执行时间}}{\text{整个任务升级后执行时间}}$$

$$\text{新执行时间} = \text{原执行时间} \times \left\{ (1 - \text{升级比例}) + \frac{\text{升级比例}}{\text{升级加速比}} \right\}$$

第一项为不能升级系统部分花费的时间，第二项为可升级部分用的时间。**升级比例**是**系统未升级前**可升级部件所用时间占总执行时间的比例。**升级加速比**是可升级部件新旧执行时间的比值

$$\text{总加速比} = \frac{\text{原执行时间}}{\text{新执行时间}} = \frac{1}{(1 - \text{升级比例}) + \frac{\text{升级比例}}{\text{升级加速比}}}$$

加速比=整个任务未升级执行时间/整个任务升级后执行时间

新执行时间=原执行时间*{ (1-升级比例) + (升级比例/升级加速比)}

升级比例是系统未升级前 可升级部件所用时间占 总执行时间的比例

升级加速比: 是可升级部件新旧执行时间的比值

$$\text{总加速比} = \frac{\text{原执行时间}}{\text{新执行时间}} = \frac{1}{(1 - \text{升级比例}) + \frac{\text{升级比例}}{\text{升级加速比}}}$$

例 假设我们希望升级一个提供Web服务的处理器。新处理器的计算速度是旧处理器的10倍。假定原处理器有40%的时间用于计算，60%的时间等待I/O操作。进行升级后，总加速比是多少？

升级比例 0.4 ， 升级加速比 10， 所以总的加速比

$$\frac{1}{0.6 + \frac{0.4}{10}} = \frac{1}{0.64} \approx 1.56$$

例 图形处理器经常需要求平方根。现有两种设计方案：第一种将浮点平方根(FPSQR)硬件加速到原来的10倍，FPSQR占据了原来总执行时间的20%，另外一种设计方案，将所有的FP浮点运算速度提高到原来的1.6倍，浮点运算占据原执行时间比例为50%。比较这两种设计方案。

通过计算加速比来比较两种设计方案

$$\text{加速比}_{\text{FPSQR}} = \frac{1}{(1 - 0.2) + \frac{0.2}{10}} = \frac{1}{0.82} = 1.22$$

$$\text{加速比}_{\text{FP}} = \frac{1}{(1 - 0.5) + \frac{0.5}{1.6}} = \frac{1}{0.8125} = 1.23$$

提高整体FP浮点运算速度的方案稍好一些，因为它的使用频率要高一些

CPU

一个程序花费的CPU时间=程序CPU的时钟周期数*周期时间

=CPU时钟周期数/时钟频率=指令条数CPI/周期时间

CPI=执行一条指令需要的时钟周期

CPI=程序的时钟周期数*指令条数

处理器性能公式

一个程序花费的CPU时间：

CPU时间=程序的CPU时钟周期数 × 时钟周期时间

CPU时间=程序的CPU时钟周期数 / 时钟频率

CPI：执行一条指令需要的时钟周期

CPI=程序的CPU时钟周期数 / 指令条数

CPU时间=指令条数 × CPI × 时钟周期时间

$$\frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}} = \frac{\text{Seconds}}{\text{Program}} = \text{CPU time}$$

一个程序时钟周期数

时钟周期时间

不同种类的指令的CPI是不一样，考虑指令种类差异之后的CPU总时钟周期：

$$\text{CPU时钟周期数} = \sum_{i=1}^n IC_i \times CPI_i$$

其中 IC_i 是第*i*类指令的数量， CPI_i 是第*i*类指令的CPI
程序的CPU时间可以表示为

$$\text{CPU时间} = (\sum_{i=1}^n IC_i \times CPI_i) \times \text{时钟周期时间}$$

总的CPI为：

$$CPI = \frac{\sum_{i=1}^n IC_i \times CPI_i}{\text{指令总条数}} = \sum_{i=1}^n \frac{IC_i}{\text{指令总条数}} \times CPI_i$$

例 假设有以下测量情况：

FP运算频率=25%，FP运算的CPI=4，其他指令的CPI=1.33

FPSQR运算(浮点平方根)频率=2%，FPSQR的CPI=20

有两种设计方案，一种方案将FPSQR的CPI降至2。另外一种将
所有FP运算CPI降至2.5，用处理器性能公式对比两种方案

仅有CPI发生改变，指令条数和时钟周期不变。每改进前的原CPI：

$$\text{原CPI} = \sum_{i=1}^n \text{CPI}_i \times \left(\frac{IC_i}{\text{指令数}} \right) = (4 \times 25\%) + (1.33 \times 75\%) = 2$$

可以用原CPI减去节省的周期数得到改进FPSQR后的CPI

$$\begin{aligned} \text{CPI}_{\text{改进FPSQR后}} &= \text{CPI}_{\text{原}} - 2\% \times (\text{CPI}_{\text{旧FPSQR}} - \text{CPI}_{\text{新FPSQR}}) \\ &= 2.0 - 2\% \times (20 - 2) = 1.64 \end{aligned}$$

对所有的FP改进后的CPI为：

$$\text{CPI}_{\text{新FP}} = (75\% \times 1.33) + (25\% \times 2.5) = 1.625$$

对所有FP改进后的CPI低一些，这种方案的效果好一些。该种方案的加速比为

$$\begin{aligned} \text{加速比}_{\text{新FP}} &= \frac{\text{CPU时间}_{\text{原}}}{\text{CPU时间}_{\text{新FP}}} = \frac{\text{IC} \times \text{时钟周期} \times \text{CPI}_{\text{原}}}{\text{IC} \times \text{时钟周期} \times \text{CPI}_{\text{新FP}}} \\ &= \frac{\text{CPI}_{\text{原}}}{\text{CPI}_{\text{新FP}}} = \frac{2}{1.625} = 1.23 \end{aligned}$$

第二章

RISC主流指令集：

RISC精简指令集

指令性质：

- 所有操作都是面向寄存器中的数据，如果数据不在寄存器中，需要先读入再进行操作
- 只有Load、Store指令访问内存
- 指令格式统一、只有少数几种指令格式
- 所有指令长度一样长

指令种类

- ALU指令——算术逻辑运算指令，比如add, subtract, and, or, xor
- Load/Store 读取/存放操作数指令
- 条件指令(branch)、跳转指令(jump)

只有load（读入操作数） store（写入存储器）指令访问内存

ALU指令：算术逻辑运算指令 add, subtract, and, or, xor

Load/Store 读取/存放操作数指令

条件指令 (branch) , 跳转指令 (jump)

ARM, RISC-v, MIPS loongArch

不同类指令执行过程

	IF	ID	EXE	MEM	WB
ALU 操作	根据PC读取指令	指令译码，读寄存器操作数	执行算术逻辑运算		结果写回寄存器堆
存储器操作 Load/ Store	根据PC读取指令		形成有效地址	Load: 从存储器读数据 Store: 把寄存器里数据写到存储器	Load: 把数据读入寄存器堆
控制类操作 条件指令 调转指令	根据PC读取指令	计算条件码和条件指令转移目标地址			

流水线性能指标:

流水线中指令相互作用

- **冒险(hazard):** 阻碍指令流中下一条即将执行指令(PC程序计数器所指向的指令)的执行
- 流水线中某条指令可能会需要被其他条指令占用的资源, 这会导致**结构冒险(structural hazard)**
- 某条指令可能会依赖前面指令的执行结果
 - 这种依赖可以是数据→**数据冒险(data hazard)**
 - 这种依赖也可以是下一条将要执行指令的地址→**控制冒险(control hazard)**, 比如条件指令(branches), 异常(exceptions)
- 解决冒险经常要在流水线中加入**停顿(stall/ bubble)**, 导致CPI>1

指令被其他指令占用资源(结构冒险), 依赖前面的执行结果(数据冒险, 控制冒险(条件待定导致))

解决数据冒险(停顿Stalling 旁路Bypass)

吞吐率 加速比 效率

分支延迟转移:

分支预测

- 虽然流水线级数增加, 分支指令的代价增加, 延迟转移分支等前面介绍的相似方法的效率不是很好满足需求
- **分支预测:** 预测分支指令/条件指令的方向——转移成功或转移不成功

- 分支预测:**
- **动态分支预测:** 根据程序行为动态预测转移方向, 转移方向动态变化
 - **静态分支预测:** 预测方向固定, 根据编译阶段的信息来预测方向, 根据早期运行情况来预测转移方向——单条分支指令往往会倾向于转移成功或转移不成功, 并非以等概率转移成功或转移不成功。
 - 任何一种分支预测方法的效果既跟方法的预测正确率有关也跟条件指令出现的频率有关。

分支预测: 预测分支指令/条件指令的方向-转移是否成功

动态分支预测：根据程序的行为动态预测转移方向，方向动态变化。预测历史记录表，2bit位预测方法

动态分支预测

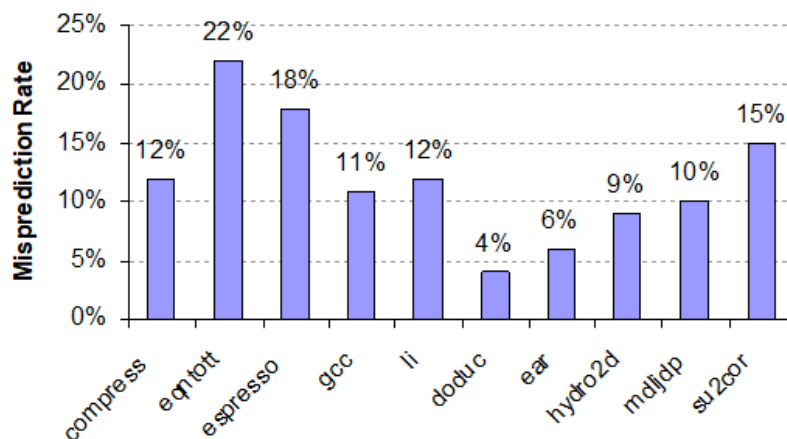
- 预测历史记录表
 - 一个容量很小存储器，用分支指令的低位地址部分来检索访问
 - 用1个标志位来记录最近该条指令转移成功与否
 - 根据地址找到匹配表项，依据历史记录预测分支转移方向
 - 通常采用2-bit/2位预测方法，而不用上述简单的1-bit/1位预测方法。

静态分支预测：预测方向固定，根据早期运行情况

静态分支预测

为了调整分支指令附近的指令，需要在编译阶段静态预测分支方向（转移成功或转移不成功），并且预测方向固定不变

最简单的方法就是，预测所有分支语句都是转移不成功——在SPEC基准测试集上，平均错误率34%



2bit分支预测方法：2个二进制位

动态 静态 转移方向

多周期5段RISC流水线分析

典型5级流水线RISC处理器

Instruction number	Clock number								
	1	2	3	4	5	6	7	8	9
Instruction i	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB

IF = instruction fetch (取指令), **ID** = instruction decode (指令译码),
EX = execution (执行指令), **MEM** = memory access (访问存储器),
and **WB** = write-back (写回结果)

不同的功能部件使用在指令执行过程的不同阶段, 减少冲突;

第三章

名称相关

名称相关:

- 名称相关: 两条指令使用相同的寄存器名称和相同的内存地址, 却没有信息流动交换。
 - 这不是一个真实的数据相关, 但是会给指令重排序造成问题。
 - 反相关 (Anti-dependence): 指令 j 要写某个寄存器或者内存单元, 而指令 i 要读相同位置或相同寄存器, 并且保持指令的初始顺序 (指令 i 在指令 j 之前)
 - 输出相关 (Output dependence): 两条指令 i 和 j 写同一个寄存器或者同一个内存单元, 需要保持指令顺序
- 由于指令间并不存在真实的信息数据交换, 可以采用重名技术来解决名称相关

反相关 (指令顺序), 输出相关 (输出时的指令顺序)

结构冒险, 控制相关:

结构冒险

- 浮点除法器没有才有流水线结构
- 寄存器端口访问频繁
 - 通过复制硬件方法来克服，寄存器堆增加多个访问端口
- 在指令译码阶段插入停顿周期
通过增加写回部件标记位来跟踪写回部件使用冲突
- 在MEM存储访问和WB结果写回阶段插入停顿周期
硬件开销少，但是停顿周期较多

结构冒险：寄存器访问频繁，（增加多个访问端口（增加写回部件跟踪（写回阶段插入停顿周期

三种相关：数据相关，控制相关，名称相关

数据相关：数据相关的指令不能同时执行（数据相关表明：存在冒险的可能性，计算结果必须遵循的顺序，可以挖掘的指令并行性上限）

- 数据相关：对于两条指令*i*和*j*,如果存在以下一种情况，则指令*j*对指令*i*存在数据依赖
 - ✓ 指令*i*执行结果会被指令*j*使用
 - ✓ 指令*j*对指令*k*存在数据依赖，指令*k*对指令*i*存在数据依赖（数据相关传递）

名称相关：用相同的寄存器名称和相同的内存地址却没有信息流动交换。

控制相关：某条指令与条件指令相关，是否会被执行取决于分支指令

数据冒险：指令间存在名称相关或者数据相关，并且指令排的比较接近就会存在风险。RAW WAW WAR三种数据冒险。

控制相关

- **数据冒险**：只要指令之间存在名称相关或者数据相关，而且指令排得比较近，指令间的重叠执行会改变对操作数的访问顺序，就会存在风险。
- 三种数据冒险：RAW写后读冒险、WAW写后写冒险、WAR读后写冒险。 **RAR读后读不是冒险。**

控制相关：就是某条指令与条件指令相关，是否会被执行取决于分支指令

- 如果一条指令与分支控制相关，就不能将该条指令移到分支指令之前，让其不再受分支指令控制。
- 如果一条指令与分支控制无关，也不能将该条指令移到分支指令之后，让其受到分支控制。

```
if (p1){s1};  
    p3=0;  
    if(p2){s2}
```

- Example 1:

```
DADDU R1,R2,R3  
BEQZ R4,L  
DSUBU R1,R1,R6  
L: ...  
OR R7,R1,R8
```

OR指令与DADDU相关，OR与DSUBU数据相关。DSUBU指令与BEQZ控制相关，不能将其移到分支指令前面

- Example 2:

```
DADDU R1,R2,R3  
BEQZ R12,skip  
DSUBU R4,R5,R6  
DADDU R5,R4,R9  
skip:  
OR R7,R8,R9
```

假设在skip后面语句中不使用寄存器R4，DSUBU指令与分支指令BEQZ不相关，可以将DSUBU移到分支指令前面

数据冒险：

WAW 写后写风险

WAR读后写风险

RAW写后读风险

相关预测：

相关分支预测

思想：记录最近的 m 条分支指令的转移方向，再利用分支指令信息（1和0组成的二进制数，记录近期若干条分支指令转移方向，比如100111，其中1表示分支指令转移成功，0表示分支指令转移不成功），选择正确的 n -bit分支转移历史表（ n -bit branch history table, 也叫 n -bit分支预测器）

gshare预测器

全局分支指令历史： m -bit移位寄存器，保存最近 m 条分支指令的转移方向，即上述的分支指令信息（体现分支间的相关性）

通常，一个 (m,n) 相关分支预测器用最近 m 条分支指令转移方向，从 2^m 个可选预测器中选中正确的预测器。每个可选的预测器都是同一条分支指令的 n -bit分支预测器，对应最近 m 条分支指令不同的转移情况。

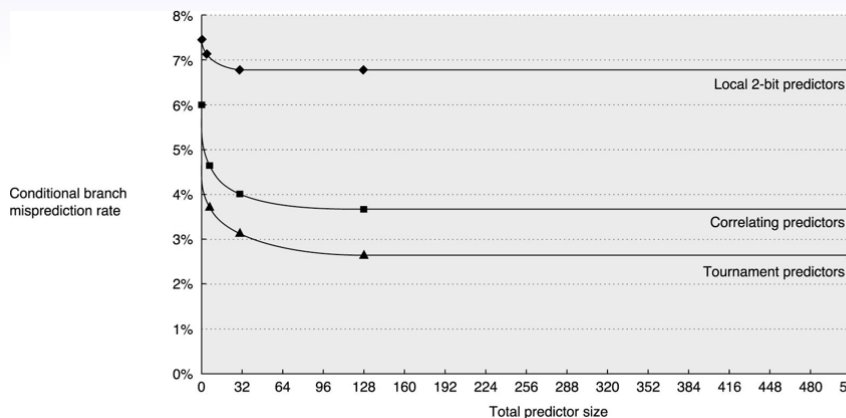
n -bit分支转移 记录最近 M 条分支指令转移方向，选中正确的预测器，每个可选的预测器都是同一条分支指令的 n -bit分支预测器，对应最近 m 条分支指令不同转移情况。

竞赛预测器：

竞赛预测器使用多个预测器，一个基于全局信息（多条分支指令转移历史），另外一个基于局部信息（某条分支指令最近若干次执行时转移情况）。再用一个选择器针对特定分支指令选择合适的预测器。

竞赛预测器使用多个预测器，一个基于全局信息，另一个基于局部信息，再用另一个选择器针对特定分支

- 竞赛预测器的好处：针对分支指令选择合适的预测器
 - 这对SPEC整数benchmark特别重要
 - 对于一个典型的竞赛预测器，在SPEC整数benchmark上，40%的时间会选择全局预测器；在SPEC浮点benchmark上，只有不到15%的时间选择全局预测器



全局 局部 选择器

全局： 2^m 次的表格使用最近 m 条分支指令的转移情况来检索

局部：一个**两级预测器**：第一级是**局部历史转移表**，用选中表项里的**二进制数检索局部历史表**。

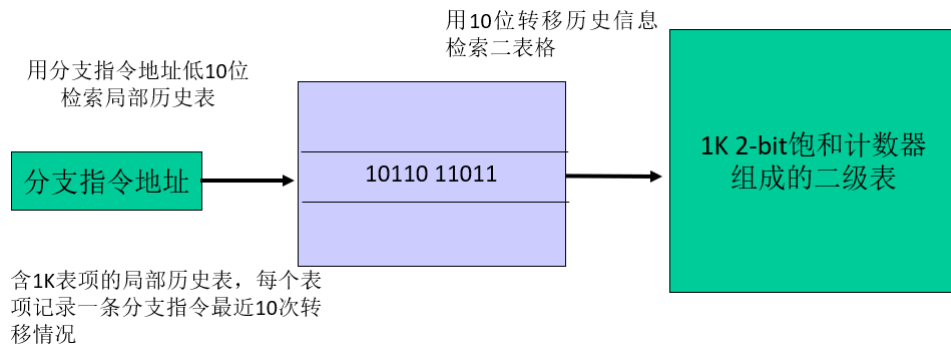
- 全局预测器

- 含有 2^m 个表项的表格，使用最近 m 条分支指令的转移情况来检索，比如 $m=10$
- 每个表项是一个标准的2-bit预测器

- 局部预测器

- 一个两级预测器。第一级是局部转移历史表，包含1024个表项。每个表项存放一个1-位的二进制数。表示一条分支指令最近10次的转移情况(1转移成功，0转移不成功)。用分支指令地址来检索局部历史表。

- 用选中的表项里的10位二进制数去检索二级表格(1K表项，每个表项是一个2-bit饱和计数器saturating counter)



Tomasulo算法指令动态调度分析例子

Tomasulo算法：**跟踪操作数，硬件方面引入寄存器重命名 最小化WAR和WAW冒险。**：

step1发射：从**FIFO指令缓冲栈**获取下一条指令，保留栈**有空闲指令**发射到保留栈中。**操作数未获得，让指令停顿。**重命名寄存器，消除WAR WAW冒险

step2执行：操作数可用，将其保存至等待该数的保留栈中。**所有操作数就位，执行指令。**防止存储器冒险，Load和store指令维持原来程序。**指令不开始执行，知道前面所有分支指令都完成**

step3写回：通过**公共数据通道CDB**将**结果写到**保留栈和寄存器堆，Store指令一直保存到store缓冲栈

动态调度——Tomasulo 算法

● 算法步骤:

➤ 发射

- ✓ 从FIFO指令缓冲栈中获取下一条指令
- ✓ 如果保留栈有空闲，将指令发射到保留栈中
- ✓ 如果操作数尚未获得，暂时让指令停顿
- ✓ 重命名寄存器，消除WAR、WRW冒险

➤ 执行

- ✓ 当操作数可用时，将其保存到正等待该数的保留栈中
- ✓ 当所有操作数都准备好了，执行指令(要避免RAW冒险)
- ✓ 为了防止存储器冒险，Load和Store指令维持原程序中的顺序
- ✓ 为了维持异常行为跟原来一样，指令不允许开始执行，一直到前面所有的分支指令都完成

➤ 写回

- ✓ 通过公共数据通道CDB将结果写到保留栈和寄存器堆
- ✓ Store指令保留在Store缓冲栈，需要一直等到接收到地址和数据

Tomasulo调度方法总结

保留栈

- ✓ 分布式检测RAW冒险
- ✓ 寄存器重命名解决WAW冒险
- ✓ 将数据(目的操作数、源操作数)缓存在保留栈，解决WAR冒险

例子详见第三章ppt

- ✓ 通过CDB进行标签匹配(Tag match)，获取需要的数据，需要增加额外硬件，进行比较

CDB公共数据通道

- ✓ 同时有多个写回时，需要多条CDB，硬件开销比较昂贵

Load/Store 读数写数缓冲栈

- ✓ 需要比较读写的内存地址，如果地址相同，需要避免冒险

方法总结:

保留栈:

1分布式检测RAW 冒险

2寄存器重命名解决WAW冒险

3数据缓存在保留栈解决WAR冒险

4通过CDB进行标签匹配

CDB公共数据通道: 同时有多个写回时，需要多条CDB，**硬件开销比较昂贵。**

LoadStore读写数缓冲栈：比较读写的内存地址，如果地址相同需要避免冒险。

带ROB的Tomasulo动态调度分析例子

含猜测Tomasulo算法步骤

➤ 1、发射——从指令队列取指令

如果保留栈和重新排序缓冲区有空闲，发射指令，送操作数以及暂存输出值的ROB编号

➤ 2、执行——执行指令

如果操作数准备好，开始执行指令。如果操作数没有准备好，监听CDB公共数据通道等待操作数。这一步需要检查RAW冒险。

➤ 3、写回

将结果通过CDB公共数据通道旁路传递给需要的功能部件和ROB

➤ 4、指令提交(instruction commit) ——用ROB里的输出结果更新寄存器和内存单元

当指令位于ROB头部(预测正确，消除不确定性)，更新寄存器和内存单元，并将指令从ROB移走。如果预测错误，将相应指令从ROB里撤销。

寄存器重命名

WAW WAR

寄存器重命名通过**保留栈**实现：若干条重叠执行指令的目的寄存器相同，**只有最后的输出值会更新寄存器堆**。

保留栈：**也可以保存（指令+操作数）**

寄存器重命名

DIV.D F0, F2, F4
ADD.D F6, F0, F8
S.D F6, 0(R1)
SUB.D F8, F10, F14
MUL.D F6, F10, F8

反相关F8

反相关F6

输出相关F6(ADD和MUL指令)

ADD和SUB之间存在WAR读后写冒险(F8)。ADD等待DIV的结果F0。DIV执行很耗费时间，SUB有可能在ADD前面完成，导致WAR冒险。

SD和MUL之间存在WAR冒险(F6)。SD等待ADD的结果F6。ADD等待DIV的结果F0。DIV执行时间远大于MUL。所以，MUL有可能在SD之前完成，导致WAR冒险。

MUL可能会在ADD之前完成，导致WAW冒险(F6)。

寄存器重命名

- 寄存器重命名通过保留栈(reservation station)实现
 - 当操作数可获得时，保留栈去取操作数，并缓存到保留栈中
 - 等待执行指令指派提供操作数的保留栈
 - 指令执行结果可以通过公共数据总线CDB(common data bus)广播传递
 - 如果若干条重叠执行指令的目的寄存器相同，只有最后的输出值会更新寄存器堆
 - 当指令被发射，用保留栈编号名称来重新命名寄存器
 - 保留栈数量会多于寄存器数量，可以去除编译器不能消除的一些冒险

处理器微结构：

顺序发射 顺序提交 乱序执行

循环展开局限性：

循环展开的局限性

- 1、随循环展开次数增加，平均每个迭代步的循环开销(计算循环变量、分支语句)减少量，变得越来越少
- 2、循环展开会让增加代码的长度，增加指令的条数
对于比较长的循环语句，这会导致Cache的不命中率增加
- 3、寄存器压力：循环展开会增加寄存器的需求数量
如果不能分配足够的寄存器，会损失一些或者全部的循环展开好处

循环展开减少了分支指令对流水线的断流影响，分支预测也实现同样目的

- 1，随着循环展开的次数增加，平均每个迭代步的循环开销减少量，变得越来越少
- 2，循环展开会增加代码的长度，增加指令条数cache不命中率增加

3, 寄存器压力: 循环展开会增加寄存器的需求数量

多发射过程:

为了是CPI小于1: 需要在在一个时钟周期里发射多条指令, 完成多条指令。

分配保留栈的同时: **限定能在一个指令束里一起发射的每一类指令的指令条数。**

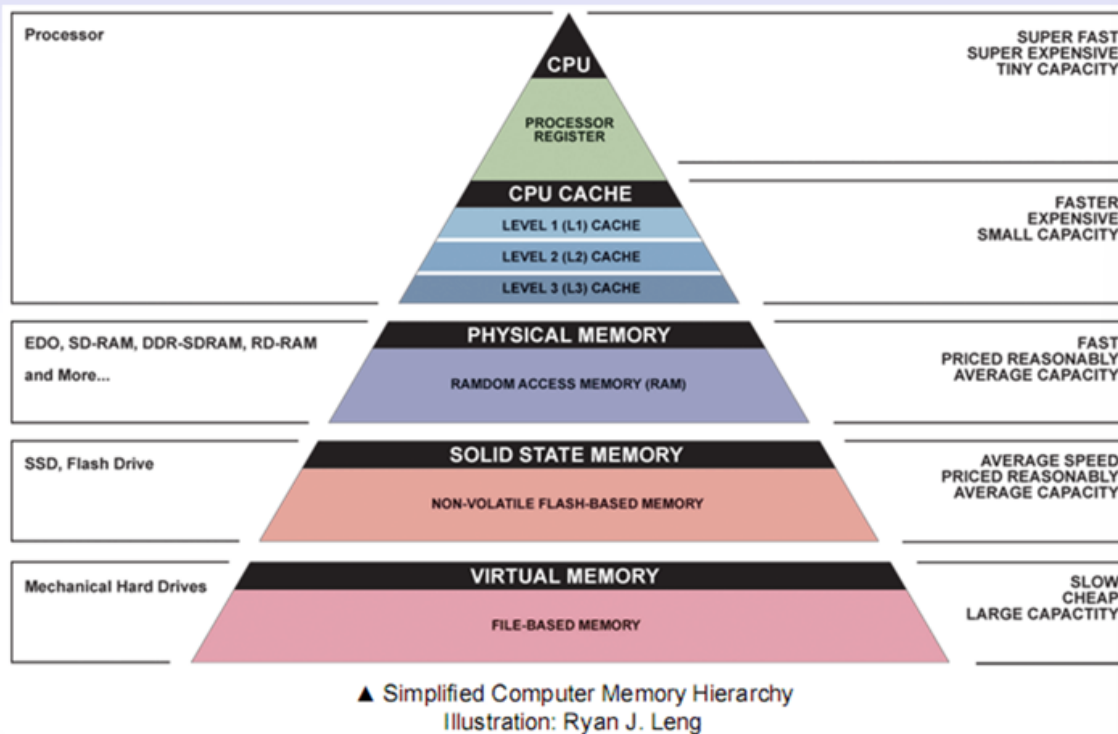
多发射过程

- 给将要发射的下一个指令束中每条指令, 分配保留栈和重排序缓冲区空间。分配保留栈时, 限定能在一个指令束里一起发射的每一类指令的指令条数, 比如一条FP运算、一条Load、一条Store、一条整型指令
- 检查指令束中指令间的相关性
- 如何指令束内指令间之间存在相关性, 用指令对应的重排序缓冲区编码来更新保留栈表, 记录相关性
- 同时也需要支持多条指令完成、多条指令提交

第四章

(四个问题) 存储器层次结构:

计算机存储器层次结构



从顶向下: CPU -> CPU CACHE -> PHYSICAL MEMORY -> SOLID STATE MEMORY -> VIRTUAL MEMORY

总峰值带宽随CPU核心增加而增加

命中: 要访问的数据在上层存储器找到

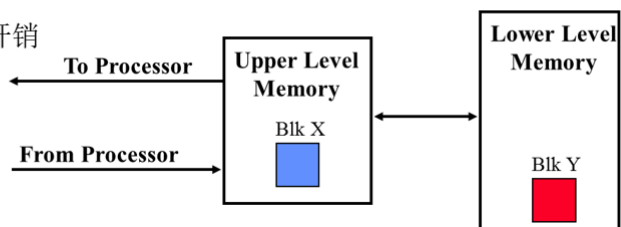
1命中率: 要访问数据在上层存储器找到的比率

2命中时间: 进入上层存储器的时间 包含进入时间+判定时间

命中时间远小于不命中时间开销

存储器层次结构基础

- **命中**: 要访问的数据在上层存储器找到(比如下图的 block X)
 - ✓ **命中率(hit rate)**: 要访问数据在上层存储器找到的比率
 - ✓ **命中时间(hit time)**: 进入上层存储器的时间, 包含进入时间+判定命中与否时间
- **不命中**: 要访问的数据不在上层存储器, 需要从下层存储器读取送到上层存储器, 再来访问, 比如block Y
 - ✓ 从下层读取数据时, 读取包含要访问数据的一整块, 根据程序局部性原理, 有利于减少接下来的数据访问不命中率
 - ✓ **不命中时间开销(Miss penalty)**: 从下层将数据替换到上层的时间+将数据送给CPU时间
- 命中时间 << 不命中时间开销



不命中原因: 首次访问某个数据块, 由于cache容量有限, 将某个数据块丢弃后又要访问数据块, 不同数据块也可以映射到同一个cache位置, 映射冲突也会不命中

- **不命中原因**:
 - ✓ 首次访问某个数据块
 - ✓ 由于Cache容量有限, 将某个数据块丢弃, 而后又要访问该数据块
 - ✓ 不同的数据块可以映射到同一个Cache位置, 映射冲突也会导致不命中
- 单条指令访存不命中 (Misses per instruction):

$$\frac{\text{Misses}}{\text{Instruction}} = \frac{\text{Miss rate} \times \text{Memory accesses}}{\text{Instruction count}} = \text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}}$$

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

命中时间

不命中时间开销

- 需要采用办法来减少访存不命中对性能的影响。带猜测机制和多线程处理器可以在发生访存不命中时, 执行其他指令

平均命中时间=命中时间+不命中率*不命中开销

1.数据块放到存储器上哪一个位置

数据块映射：将数据块block12放到有8个块的cache：全相联，直接相联，组相联

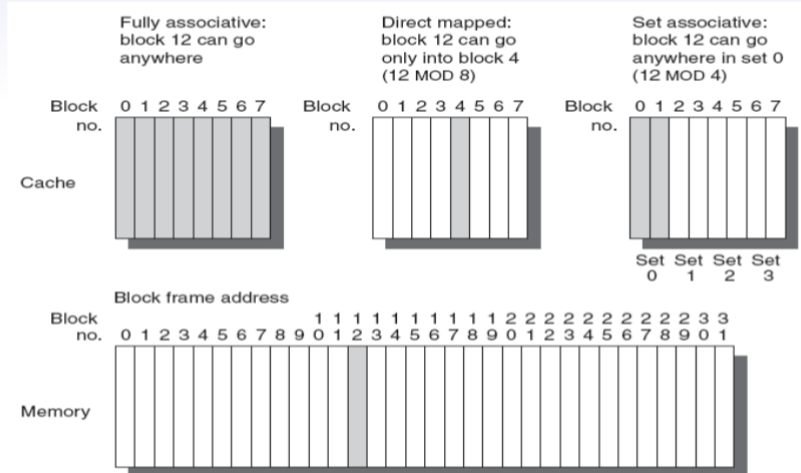
问题1：数据块映射

将数据块Block 12放到有8个块block的Cache

——全相联Fully associative(随意映射)、直接相联direct mapped(取模运算)、组相联n-way set associative

——组相联，内存块与Cache组间采用直接相联

Block number MOD Number of sets，内存块在组内采用全相联



2.如何在上层存储器中找到数据块

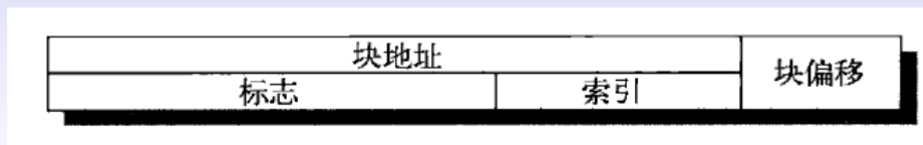
数据查找：直接相联或者组相联Cache中的地址组成。（标志+索引+块内偏移）

标志tag：通过比较标志字段，检查组内是否有匹配块（是否命中）

索引index：用来选择组别（全相联没有这项）

块偏移：块内偏移地址，确定需要访问数据在cache块中的位置

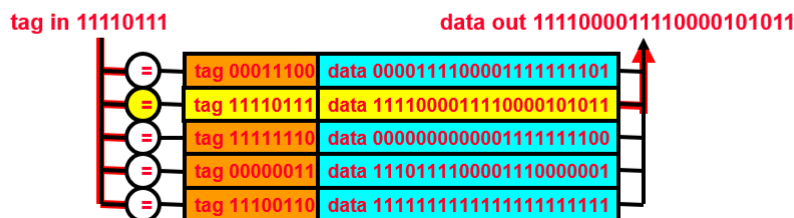
问题2：数据查找



- 直接相联或者组相联Cache中地址组成：
 - ✓ 标志tag：通过比较标志字段，检查组内是否有匹配块(是否命中)
 - ✓ 索引index：用来选择组别，全相联没有这项
 - ✓ 块偏移block offset：块内偏移地址，确定需要访问数据在Cache块内的地址

全相联查找

- 每个block都要进行标志tag比较，为了速度需要并行比较。
- 需要给每个 block配置一个比较器，用于标志比较
- 不需要地址译码(没有索引字段，只有标志字段)
- 全相联只适用于小容量Cache



3.当发生数据不命中，替换哪一个数据块

对于直接相联，利用模运算直接映射

对于组相联和全相联：

随机算法：随机选择一块被替换

LRU：近期最久没用的块替换

FIFO：先进的先被替换

问题3：数据块替换

- 对于直接相联，利用模运算直接映射
- 对于组相联和全相联：
 - ✓ 随机算法：随机选择一块被替换
 - ✓ LRU(近期最久没用到)算法：基本思想是近期被用到的块大概率还会被再次使用，所以选择近期最久没用的块当做被替换的块
 - ✓ FIFO先进先出算法：最先进来的块被替换

不同cache块的大小，在各种算法下的性能也不一样。

相同cache大小：相联度增加不命中次数下降；对于大尺寸Cache，LRU与随机算法性能差不多。但是对于小尺寸Cache，LRU性能优于其他两种算法。FIFO优于随机算法

4.如何处理数据更新：

更新策略：写直达法 更新Cache数据时，同步更新内存数据；

写回法：当cache块被替换时更新相应的数据

问题4

- Cache数据更新策略：
 - ✓ 写直达法write-through：在更新Cache数据时，同步更新内存数据
 - ✓ 写回法write-back：只有当Cache块被替换时，才更新内存的相应数据
- 两种策略比较：
 - ✓ 1、如果数据改变多次，写直达法多次会写内存，写回法则不会。
 - ✓ 2、写直达法与内存通信量大，写回法与内存通信量小，比较适合嵌入式应用
 - ✓ 3、写直达法能较好保持数据一致性，更加便宜实现多级Cache。因为只需要保持与相邻级Cache数据一致，不用一直追溯到内存。
 - ✓ 4、写直达法实现比较容易

比较：

如果数据改变多次，写直达就会多次写内存 写回法不会；

写直达与内存通信量大，写回法与内存通信量小，适合嵌入式。

写直达法更便宜实现多级cache 保持数据一致性。

写直达法容易实现

可以利用写缓冲器优化写直达

存储墙

Cache高级优化方法:

Cache高级优化方法

考虑Cache不命中后，CPU运行时间

$$CPUtime = IC * (CPI_{Execution} + \frac{MemoryAccess}{Instruction} * MissRate * MissPenalty) * ClockCycleTime$$

- 减少命中时间
小容量L1 Cache; 组内块(路)预测
- 增加Cache带宽
流水线结构Cache; 多缓存组Cache; 无阻塞Cache
- 减少不命中时间开销
关键字优先; 合并写缓冲器
- 减少不命中率
编译器优化
- 利用并行性减少不命中时间开销和不命中率
硬件预取; 编译器预取

高级优化方法:

- 1.命中时间小容量L1 cache 组内块路预测
- 2.增加cache带宽: 流水线结构cache 多缓存组cache 无阻塞cache
- 3.减少不命中时间开销: 关键字优先: 合并写缓冲器
- 4.减少不命中率: 编译器优化
- 5.利用并行性减少不命中时间开销和不命中率: 编译器预取

提高Cache带宽: 分组结构, 无阻塞 流水线结构

无阻塞cache提高带宽: 当cache不命中发生时, 不停顿, 无阻塞允许正当发生不命中时继续提供cache命中进行 后续命中

L2级cache需要支持无阻塞

多组结构: 将cache组织分成多个独立的, 支持同时访问的缓存组

将Cache访问流水线化提高带宽

- 将Cache访问流水线化，可以分成多个流水步，在多个时钟周期内完成，从而可以采用更快的时钟周期，提高Cache的带宽

例如：

Pentium: Cache访问要 1个周期

Pentium Pro--- Pentium III: Cache访问要 2个周期 2个流水步

Pentium 4---Core i7: Cache访问要 4个周期 4个流水步

- Cache访问流水化使得增加Cache相联度更容易，但是会增加分支预测错误的时间开销

用无阻塞Cache提高带宽

- 对于采用乱序执行的流水线处理机，当发生Cache不命中时，不需要因为Cache不命中而停顿。无阻塞Cache允许正当发生不命中时，继续提供Cache命中，也就是在不命中没处理完成前允许后续命中进行。
- L2二级Cache需要支持无阻塞。一般而言，处理器可以隐藏L1级的不命中时间开销，但不能隐藏L2的不命中时间开销。

非阻塞Cache的有效性评估(在一个不命中发生时允许后续命中1次、2次或者64次)

在Miss时允许1次命中，可以让整型基准集整体不命中惩罚(不命中时间开销)减少9%，浮点基准集整体不命中开销减少12.5%。如果允许2次命中，则将不命中开销减少10%和16%。允许64次命中仅仅带来少量额外减少。

多组结构Cache提高带宽

- 将Cache组织成多个独立的、支持同时访问的缓存组 (multi-bank)
 - ✓ ARM Cortex-A8的L2有1-4个缓存组 (bank)
 - ✓ Intel i7的L1有4个缓存组，L2有8个缓存组

Block address	Bank 0	Block address	Bank 1	Block address	Bank 2	Block address	Bank 3
0		1		2		3	
4		5		6		7	
8		9		10		11	
12		13		14		15	

Figure 2.6 Four-way interleaved cache banks using block addressing. Assuming 64 bytes per blocks, each of these addresses would be multiplied by 64 to get byte addressing.

4路交错缓存组

Cache映射:

直接相联 组相联 全相联 (见前面)

Cache替换方法:

FIFO LRU 随机 (见前面)

Cache更新方法比较

写回法 写直达法 (见前面)

第五章

挖掘数据级并行的三种结构:

GPU向量机

SIMD多媒体扩展

GPU的调度器:

线程调度器, 线程块调度器

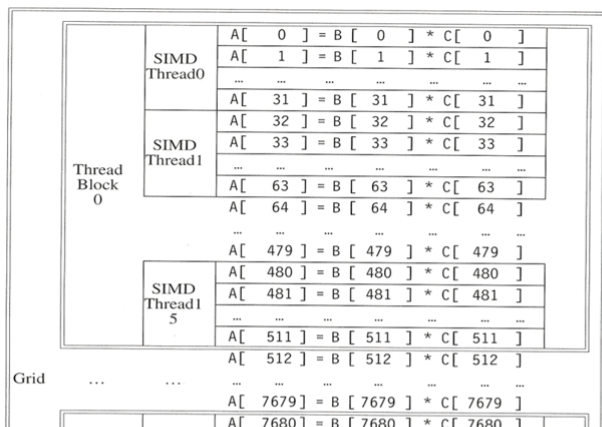
线程：一个SIMD指令序列 线程组成线程块block 线程块组成网格grid

线程thread和线程块block

- 线程：一个的SIMD指令序列
 - ✓ 数以千计的线程可用于各种并行性：多线程、SIMD、ILP、MIMD
- 线程可以组成线程块block
 - ✓ 线程块最多可以处理512个元素，每个线程同时处理32个元素，线程块包含16个线程
 - ✓ 多线程SIMD处理器：执行整个线程块的硬件
- 线程块组成网格(grid)
 - ✓ 线程块可以按任意顺序独立执行
 - ✓ 不同的线程块不能直接通信，但是可以通过全局存储器和存储器原子操作(atomic memory operation)来协同工作。
- GPU硬件来处理线程管理，不是应用程序或者操作系统
 - ✓ 多处理器由多线程SIMD处理器组成
 - ✓ 线程块调度器thread block scheduler: 将线程块调度分配给不同的SIMD处理器

示例

- 两个长度为8192的向量相乘
 - ✓ 网格grid: 执行所有元素操作的代码
 - ✓ 线程块将网格分成可管理的尺寸
 - 512个元素/线程块，16个SIMD线程/线程块→32个元素/线程
 - ✓ SIMD指令一次执行32个元素操作
 - ✓ 网格大小=16个线程块
 - ✓ 通过线程块调度器thread block scheduler,将线程块分配给多线程SIMD处理器
 - ✓ Fermi GPU有7-15个多线程SIMD处理器



一个线程操作32个向量元素

16个线程/线程块

SIMD 多媒体扩展

- 多媒体应用的数据类型长度往往会小于字长
 - ✓ 图形系统使用8位来表示三个原颜色比如`rgb`
 - ✓ 声音采样通常用8位或者16位表示
- 如果一个处理器有一个256位加法器，则它可以同时处理32个8位操作数，或者16个16位操作数，或者8个32位操作数，或者4个64位操作数

典型256位多媒体扩展操作

Instruction category	Operands
Unsigned add/subtract	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Maximum/minimum	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Average	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Shift right/left	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Floating point	Sixteen 16-bit, eight 32-bit, four 64-bit, or two 128-bit

- 与向量指令相比较，SIMD多媒体扩展：
 - 将操作码中操作数数量固定，导致在X86结构的多媒体扩展集MMX，SSE和AVX中增加了几百条指令。
 - 没有复杂的寻址方式(比如步幅访问、集中分散访问)
 - 没有掩模寄存器用于条件执行向量元素。

向量机

- 基本思想：
 - ✓ 将数据元素集合读入向量寄存器
 - ✓ 操作基于向量寄存器
 - ✓ 将结果分散写回存储器
- 向量寄存器受到编译器控制
 - ✓ 用于隐藏存储器延迟
 - ✓ 以杠杆作用方式扩大存储器带宽

SIMD的优势：

- 1很容易实现，只要对标准ALU增加些许开销
- 2只需要少许额外状态-易于上下文转换context Switch
- 3只需要增加少许的额外带宽
- 4没有虚拟存储器的页面错误，跨页访问等问题
- 5易于引入新的指令，支持新的多媒体数据标准

屋脊线Roofline性能模型

运算吞吐量随算数密度变化的曲线，将浮点性能 存储器性能 算术强度关系反映到2d图形中
是一种比较各种SIMD体系结构性能的**直观可视化方法**。

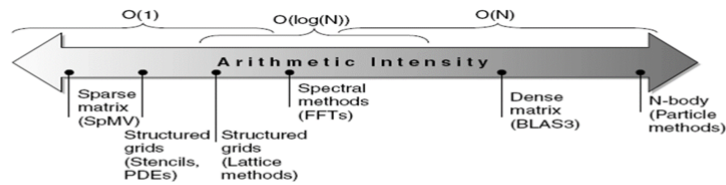
屋脊线Roofline性能模型

➤ 基本思想

- ✓ 画出峰值浮点运算吞吐量随算术密度变化的曲线
- ✓ 将浮点性能、存储器性能、算术强度关系反映到2D图形中
- ✓ 这是一种比较各种SIMD体系结构性能的直观可视化方法

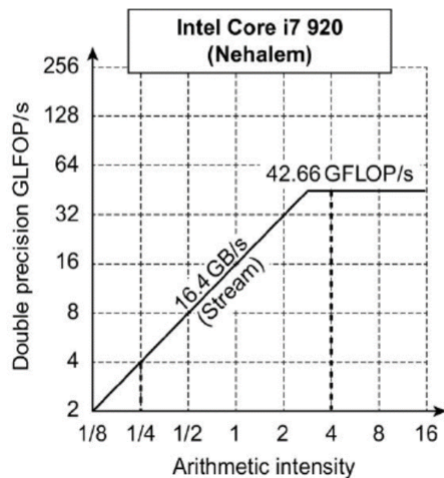
➤ 算术密度Arithmetic intensity

- ✓ 每访问存储器一个字节，包含的算术运算量

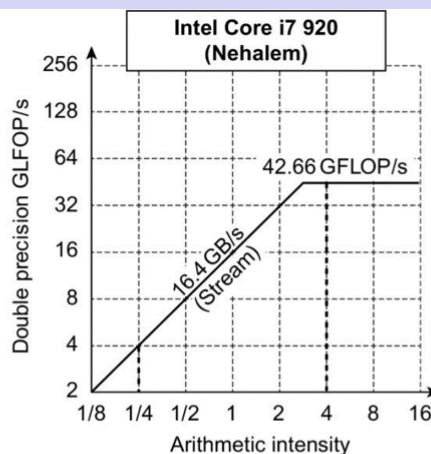
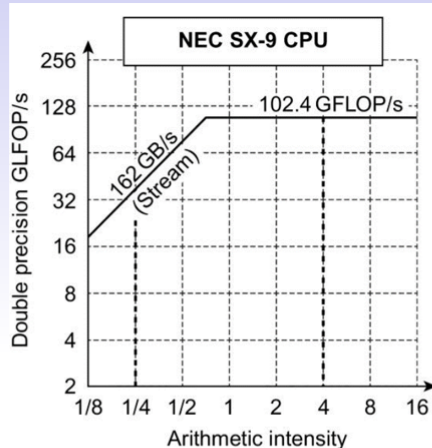


几种情况下的算术密度

可获得的 GFLOP/s = Min (峰值存储器带宽 × 运算密度, 峰值浮点性能)



- 屋脊线的倾斜部分表示性能受到存储器带宽的限制。斜线的斜率就是峰值存储带宽
- 对于屋脊线的水平部分，表示性能受限于峰值浮点运算(处理器硬件限制)
- 背脊点ridge point是水平线与倾斜线的交叉点。
- 如果该点很靠近右边，只有很高算术密度的核心代码集才能达到最高的运算性能；如果该点很靠近左边，那么几乎所有的核心代码集都能达到最高性能。



SX-9的峰值计算性能是Core i7的2.4倍。 SX-9存储器性能Core i7

是10倍。当运算密度为0.25，SX-9运算性能是Core i7的10倍(40.5 vs 4.1 GFLOP/sec)。SX-9的背脊点为0.6，Core i7的背脊点为2.6。这表明有更多的程序可以在向量机上SX-9获得最大性能。

向量机：

向量机

- 基本思想：
 - ✓ 将数据元素集合读入向量寄存器
 - ✓ 操作基于向量寄存器
 - ✓ 将结果分散写回存储器
- 向量寄存器受到编译器控制
 - ✓ 用于隐藏存储器延迟
 - ✓ 以杠杆作用方式扩大存储器带宽

基本思想：将数据元素集合读入向量寄存器；

操作基于向量寄存器；

将操作结果分散写回寄存器；

向量寄存器收到编译器控制：用于隐藏存储器延迟，以杠杆作用方式扩大存储器带宽。

向量机结构总结

- 通道multiple lanes: 一个时钟周期处理多于1个元素
- 向量长度寄存器：处理向量长度不为64位的情况
- 向量掩码寄存器：向量代码中包含有IF语句，只处理部分满足条件元素处理
- 存储器组块：优化存储系统，以支持向量处理器
- 步幅stride: 处理向量体系结构中的多维矩阵
- 集中-分散：处理向量体系结构中的稀疏矩阵(很多元素为0)
- 向量体系结构编程：编译器反馈给程序某段代码是否可以向量化，程序员可以给编译器提示

向量机结构：**通道**，**向量长度寄存器**，**向量掩码寄存器**，**存储器组块**，**步幅stride**，**集中-分散**，向量体系结构编程。

集中-分散操作

集中操作LVI：**使用索引向量，取到稀疏矩阵中非零元素**

分散操作SVI：**当以紧凑形式完成操作处理后，稀疏向量可以采用相同的索引向量，扩展会原来的分散模式**

分散与集中

- 稀疏矩阵需要按照向量模式执行。在稀疏矩阵中，向量元素通常用某种紧凑形式存放，然后使用非零元素索引来间接访问。
- 集中分散操作gather-scatter operation:
 - ✓ 集中操作：使用索引向量，取到稀疏矩阵中非零元素
---LVI (*load vector indexed or gather*)
 - ✓ 分散操作：当以紧凑形式完成操作处理之后，稀疏向量可以采用相同的索引向量，扩展存回原来的分散形式
---SVI (*store vector indexed or scatter*)
- 分散与集中操作支持稀疏矩阵在紧凑压缩形式(不包含非零元素)和正常形式(包含非零元素)之间转换。

第六章

消息传递系统，共享存储器系统

并行结构分为消息传递系统和共享存储系统

消息传递系统：每个处理器有自己的私有存储器，处理器通过显示消息通信

共享存储系统：多个处理器共享一个存储器，通信通过共享存储器实现。

并行结构

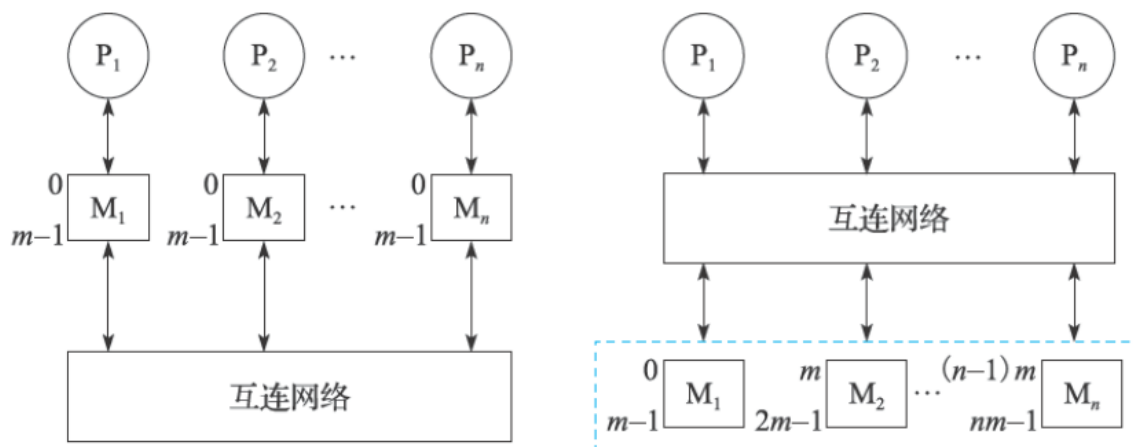


图 10.3 消息传递（左）和共享存储系统（右）

并行结构，分为消息传递系统(MPP、Cluster)和共享存储系统(SMP、DSM)

消息传递系统：每个处理器有自己的私有存储器，处理器间通过显式消息通信

共享存储系统：多个处理器共享一个存储器，通信通过共享存储器实现

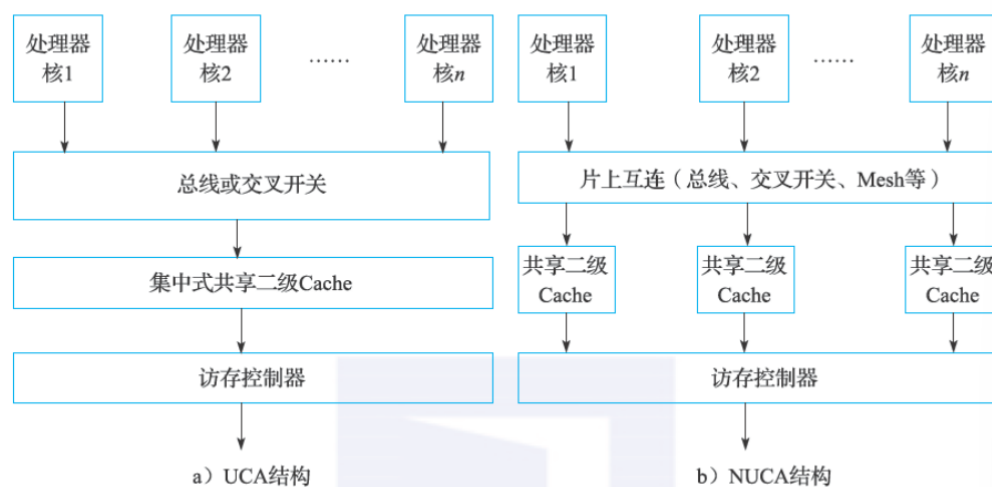
多核处理器与共享内存多处理器相似，一个核运行一个线程

LLC

UCA结构：集中式共享LLC，cache访问延迟一样 扩展性差 适和核数较少的情况

NUCA结构：分布式共享LLC 访问延迟不一样，需要高效替换算法，扩展性强，适和核数较多

共享LLC



UCA结构：集中式共享LLC，Cache访问延迟一样，扩展性弱，适合核数较少的情况

NUCA结构：分布式共享LLC，访问延迟不一样，需要高效查询替换算法，扩展性强，适合核数较多和众核(>64核)的情况

cache一致性

Cache一致性：在共享存储器系统中，维持数据在存储器和多个处理器或多个核的私有Cache中的数据副本一致

在共享存储器系统中，维持数据在存储器和多个处理器或多个核的私有cache中的数据副本一致。

监听协议和目录协议，适用场景，优劣

Cache 一致性协议的分类

- 从新值要传给谁，可以分为：
 - 监听协议Snoopy protocol
 - ✓ 当某个处理器写一个共享数据时，**将写无效信号通过信道广播给所有的处理器**。每个处理器监听信道，看无效信号是否与自己的Cache副本有关，如果是则将副本作废。
 - ✓ 常用于SMP系统，需要广播，适合于共享总线结构。
 - ✓ 总线是独占共享资源，延迟会随处理器数量增多而增加，限制了连接的处理器数量
 - 目录协议Directory protocol
 - ✓ 为每各存储行维持一目录项，**记录所有当前持有此行数据备份的处理器号以及是否已被改写等信息**。
 - ✓ 当一个处理器核写数引起数据不一致时，它根据目录的内容**只向持该数据备份的处理器发出写无效/信号**，避免了广播。
 - ✓ 扩展性比较好，可以连接较多数量的处理器，多用于DSM系统

监听协议：总线控制权获得的顺序性保证多个处理器对同一数据写操作的串行化，所有一致性协议都需要保证对统一cache写操作的串行化。

共享位1表示有多个副本 0表示被独占

- 写操作时，如果共享位0，表示数据只有唯一副本，不需要将作废消息发到总线，减少带宽，加快执行速度；
- 写操作时，如果共享位1，需要发送作废消息，将其他Cache数据副本作废。同时，被写Cache共享位设为0，表示为拥有者。如果独占数据后面被复制，共享变为1。

场景举例：写作废协议 MSI (I无效 S共享 M修改)

目录协议：依托目录表项，避免广播操作。存储器和目录表分布到各个节点。每个节点目录表只记录该节点的存储器的信息

场景举例：cache块的状态转移 和 目录表中存储块的状态转移

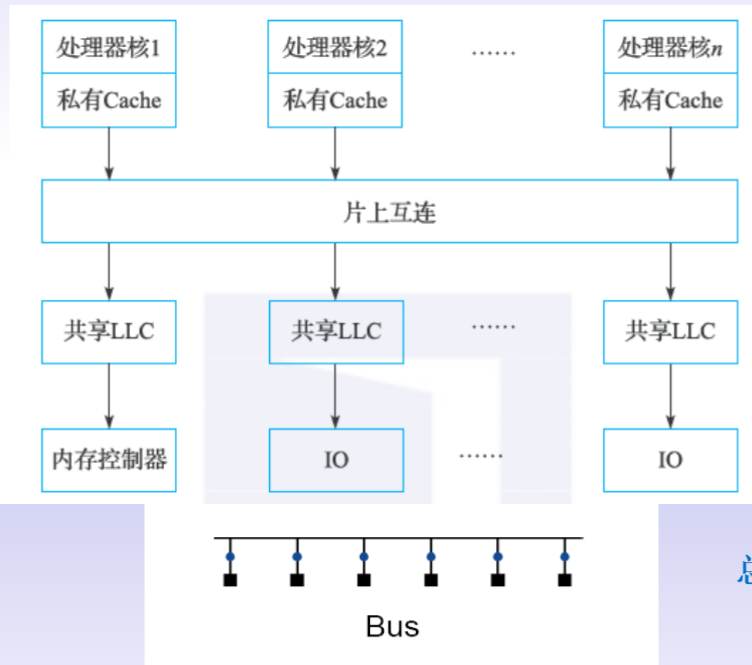
互连网络

多核处理器通过互连网络将处理器等链接起来

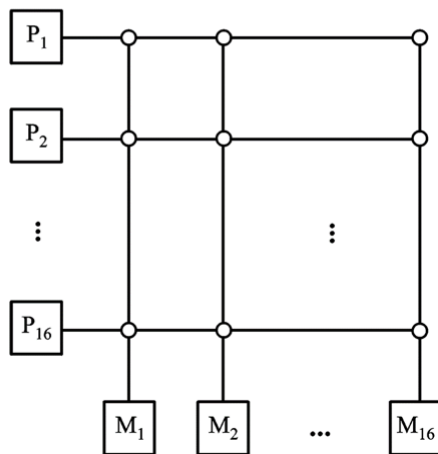
总线 交叉开关 环 网格

互连网络

多核处理器通过互连网络将处理器核、Cache、内存控制器、IO 接口等模块连接起来。



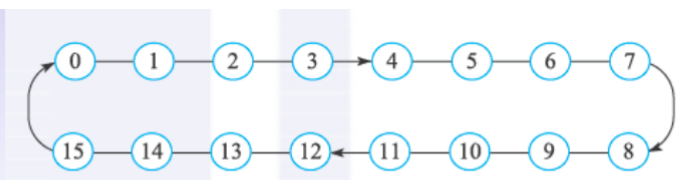
交叉开关crossbar



交叉开关：以矩阵形式组织的开关集合。每一个输入能连到任意输出。

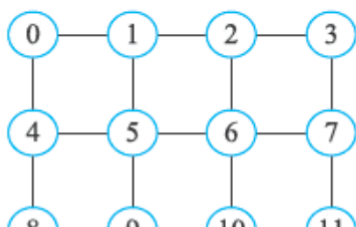
交叉开关有多个输入线和输出线，这些线交叉连接在一起，交叉点可以看作单个开关。当一个输入线与输出线的连接点处开关导通时，则在输入线与输出线之间建立一个连接。

比如P₁和M₁交叉开关导通时，它们建立连接。同时P₂与M₂也可以建立连接。两组连接平行传输。

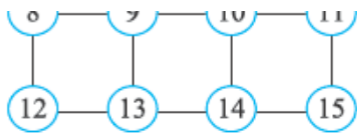


环Ring

网格Mesh



源节点到目的节点传递需要进行路径选择。由路由算法来完成。需要考虑网络负载平衡问题



硬件多线程:

允许多个线程以重叠执行方式共享单个处理器的功能部件以提高资源利用效率。

MIMD依赖多个进程和线程让处理器处于忙碌

支持多线程需要：复制每个线程的状态，支持线程快速切换

硬件多线程

- **硬件多线程(Hardware multithreading)**
 - 允许多个线程以重叠执行方式共享单个处理器的功能部件，以提高硬件资源的利用效率。
- MIMD依赖多个进程和线程让多个处理器处于忙碌状态。
- 为了支持硬件多线程，处理器需要：
 - 复制每个线程的状态，比如每个线程需要有独立的寄存器堆、PC。
 - 支持线程的快速切换(线程切换远远快于进程切换)

粗粒度多线程

1只有遇到长停顿，比如最后一集cache不命中才切换进程

2减少进程切换频率，加快单个进程执行简化硬件

3不能隐藏短停顿 比如数据相关

4流水线清空装入开销大

粗粒度多进程Course-grain Multithreading

- 只有遇到长停顿，比如最后一级Cache不命中，才切换进程
- 减少进程切换频率，加快单个进程执行，简化硬件
- 不能隐藏短停顿，比如数据相关
- 流水线清空装入开销大。

细粒度多线程

1每个时钟周期切换一次进程，多线程指令交替执行

2如果某个进程出现停顿，更换另一个进程

3可以隐藏流水香执行中长短停顿，但是会推迟单个进程执行

细粒度多线程Fine-grain Multithreading

- 每个时钟周期切换一次进程。多线程指令交替执行。
- 如果某个进程出现停顿，更换另外一个进程。
- 可以隐藏流水线执行中长短停顿，但是会推迟单个进程执行